

Universidad Nacional Autónoma de México

FACULTAD DE CIENCIAS

Ciencias de la Computación



Criptografía y Seguridad

Práctica 1

Cifrados Clásicos

Aguirre Cruz Cassandra -

González Téllez Lorena - 321288952

Vázquez Rincón Oscar - 321228954

1 de septiembre de 2026

1. Introducción

La criptografía es la ciencia encargada de diseñar funciones o dispositivos capaces de transformar mensajes legibles o en claro a mensajes cifrados de tal manera que dicha transformación (cifrar) y su transformación inversa (descifrar) sólo pueden ser factibles con el conocimiento de una o más llaves. En la criptografía existen dos tipos de cifrados: los simétricos, en donde la misma llave se usa para cifrar y descifrar, y los asimétricos, en donde se utilizan dos llaves distintas. Esta práctica se centra en los cifrados simétricos.

Los cifrados clásicos son aquellos que han tenido un uso histórico masivo, pero que hoy en día no se consideran seguros, pues han surgido técnicas modernas más sofisticadas a lo largo del tiempo.

Esta práctica tiene como objetivo implementar algunos de estos cifrados clásicos y aplicarlos para el cifrado y el descifrado de archivos. El desarrollo de la misma implica trabajar directamente a nivel de bytes y realizar corrimientos correspondientes según el cifrado empleado. El enfoque se centra en el cifrado César, el cifrado decimado, el cifrado afín y la codificación Base64. Para poder descifrar los archivos sin conocer la llave de antemano, se hace uso de las firmas de archivo o Magic Bytes. Se busca combinar este conocimiento con las propiedades matemáticas de los cifrados para deducir la llave correcta y recuperar los archivos originales.

La importancia de esta práctica radica en comprender la base y los cimientos de lo que es la criptografía moderna. Saber aplicar estos cifrados clásicos permite construir bases sólidas sobre el origen de la disciplina, aplicando congruencias y aritmética modular. Por otro lado, el enfrentarse a un escenario de descifrado, como el que plantea esta práctica, nos permite desarrollar conocimientos sobre el manejo de bytes y las firmas de archivo.

2. Desarrollo

El desarrollo de esta práctica consta de distintas etapas: los cifrados, las firmas de los archivos y el manejo de bytes.

2.1. Cifrados

Base64

Correspondiente a la codificación Base64 se implementó el archivo `base64_codec.py`, el cual funciona como codificador y decodificador Base64. Primero, la función `base64_encode` se encarga de transformar los datos originales en su representación Base64, en nuestra implementación se espera operar sobre la secuencia de bits ya representada como cadena de texto. Es importante notar que la información se codifica en bloques de 6 bits y de ser necesario se reagrupa la secuencia completa de bits para poder mapear cada bloque a un valor numérico entre 0 y 63, que luego se traduce a un carácter del alfabeto definido (la variable `CARACTERES`).

Además, los caracteres sobrantes al final se sustituyen por '=' que fueron los de relleno en la reorganización de la secuencia de bits.

Por otro lado, la función `base64_decode` hace lo inverso y permite recuperar los datos originales. Contrario a la codificación, se eliminan los '=' y para cada carácter válido se encuentra su posición dentro del alfabeto definido para Base64 y se traduce a su equivalente de seis bits. Estos bloques se concatenan de forma que constituyen la cadena binaria completa. Nuevamente se reagrupan para formar la secuencia de bytes original (que constan de 8 bits y no 6).

Ambas funciones trabajan de manera que deben garantizar que la información se transforme y recupere de manera íntegra.

```
def base64_encode(data: bytes) -> str:
    """
    Codifica datos a Base64

    Args:
        data: Datos a codificar

    Returns:
        Datos codificados a Base64
    """
    CARACTERES = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    completo = len(data) % 3
    p = 0 if completo == 0 else (3 - completo)
    while len(data) % 6 != 0:
        data += "\0"
    res = ""
    for i in range(0, len(data), 6):
        bloque = data[i:i+6]
        indi = int(bloque, 2)
        res += CARACTERES[indi]
    if p > 0:
        res += "-" * p
    return res
```

(a) Codificación Base64

```
def base64_decode(encoded: str) -> bytes:
    """
    Decodifica datos de Base64

    Args:
        encoded: Datos codificados a Base64

    Returns:
        Datos decodificados
    """
    word = encoded.replace("=", "")
    CARACTERES = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    res = ""
    for char in word:
        if char in CARACTERES:
            index = CARACTERES.index(char)
            res += format(index, '06b')
    while len(res) % 8 != 0:
        res = res[::-1]

    result = b""
    for i in range(0, len(res), 8):
        byte = res[i:i+8]
        if len(byte) == 8:
            result += bytes([int(byte, 2)])
    return result
```

(b) Decodificación Base64

Figura 1: Proceso de codificación y decodificación en Base64.

2.2. Firmas de Archivos

Para esta etapa de la práctica se implementó el archivo `magic_bytes.py` que tiene como propósito la identificación del formato real de un archivo a partir de su contenido binario. Dado que las extensiones de los archivos pueden ser modificadas, los formatos binarios incluyen al inicio del archivo una secuencia fija de bytes que facilitan la identificación.

Definimos un diccionario `MAGIC_BYTES_HEX` que mapea la representación hexadecimal de cada firma a su formato. Mediante la función `detectar_formato` comparamos el contenido del archivo con las firmas conocidas. Ya sea que se reciba un flujo de datos o una secuencia de bytes, se leen únicamente los primeros 16 bytes del archivo, los cuales pueden contener cualquiera de las firmas registradas.

```
MAGIC_BYTES_HEX = {
    "89504e470d0a1a0a": "png",
    "ffd8ff": "jpg",
    "424d": "bmp",
    "494433": "mp3",
    "fffb": "mp3",
    "4f676753": "ogg",           # OggS
    "0000001866747970": "mp4",
    "1a45dfa3": "mkv",
    "504b0304": "docx",         # PK.. (También zip, epub, xlsx...)
    "25504446": "pdf",         # %PDF
    "65707562": "epub",        # epub
}

RIFF = "52494646"
```

Figura 2: Diccionario de firmas de archivos en `magic_bytes.py`.

```
def detectar_formato(data: bytes) -> str | None:
    """
    Determina el formato de un archivo a partir de su contenido binario.

    Params:
    | data: Contenido binario del archivo

    Returns:
    | Formato del archivo como string o None si no se puede determinar.
    """
    if hasattr(data, 'read'):
        start_pos = data.tell() if hasattr(data, 'tell') else 0
        bytes_data = data.read(16)
        if hasattr(data, 'seek'):
            data.seek(start_pos)
    else:
        bytes_data = data[:16]

    hexbytes = bytes_data.hex()

    # Para el caso de RIFF, WAV y AVI comparten los primeros 4 bytes.
    # Los bytes 8-11 indican el subtipo sea wav o avi
    if hexbytes.startswith(RIFF) and len(bytes_data) >= 12:
        subtype = bytes_data[8:12]
        if subtype == b"WAVE":
            return "wav"
        if subtype == b"AVI ":
            return "avi"
        return None # tipo desconocido

    for magic_hex, formato in MAGIC_BYTES_HEX.items():
        if hexbytes.startswith(magic_hex):
            return formato

    return None
```

Figura 3: Implementación de la función `detectar_formato`.

Una observación importante es que los formatos WAV y AVI comparten los mismos 4 bytes iniciales, 52 49 46 46, por lo que no es posible distinguirlos únicamente con el encabezado principal. Para ello, cuando se detecta la firma RIFF se inspeccionan los bytes 8 a 11 del archivo y se identifican correctamente estos formatos.

Por otro lado, `es_archivo_valido` fue implementada para verificar si un archivo corresponde a un formato específico. En lugar de indicar qué formato fue detectado, devuelve un valor booleano que confirma si el resultado coincide con el formato esperado (esto porque no podemos confiar de la extensión que vemos superficialmente).

```
def es_archivo_valido(data: bytes, formato_esperado: str) -> bool:
    """
    Verifica si un archivo es del formato esperado

    Params:
        data: Contenido binario del archivo
        formato_esperado: Formato esperado

    Returns:
        True si el archivo es del formato esperado, False en caso contrario
    """
    formato_detectado = detectar_formato(data)
    return formato_detectado == formato_esperado
```

Figura 4: Implementación de la función `es_archivo_valido`.

2.3. Manipulación de bytes

Por último, `archivos_bytes.py` fue implementado para manejar las operaciones de entrada y salida a nivel de bytes, sirviendo de soporte para los cifrados, la codificación Base64 y la detección de formatos. Define tres funciones:

- `leer_archivo(ruta)`: Verifica que el archivo exista en la ruta indicada y, de ser así, lo abre en modo binario y devuelve su contenido completo como una secuencia de bytes.
- `escribir_bytes_archivo(ruta, contenido)`: Recibe una secuencia de bytes y la escribe en el archivo de destino. Si el directorio especificado en la ruta no existe, lo crea automáticamente antes de escribir.
- `guardar_descifrado(descifrado, nombre, extension)`: Construye la ruta de salida con el formato `<nombre>_descifrado.<extension>`, invoca a `escribir_bytes_archivo` para guardar el contenido y retorna la ruta completa del archivo generado.

2.4. Flujo de Trabajo

Como resultado de las implementaciones anteriores, se diseñó el programa `main.py` que integra todos los módulos para intentar descifrar un archivo cuyo método de cifrado y llave son desconocidos. El programa aplica un enfoque de fuerza bruta, esto es, prueba distintos algoritmos de descifrado sobre el contenido binario del archivo y, tras cada intento, verifica si el resultado tiene una firma de archivo reconocida.

Funciones auxiliares

Para mantener el código organizado, se definieron funciones auxiliares que encapsulan tareas repetitivas:

- **guardar(datos, nombre_base, formato):** Crea el directorio de salida si no existe y escribe el archivo descifrado asignándole el nombre `<nombre_base>_descifrado.<formato>`. Retorna la ruta del archivo guardado.
- **verificar(datos):** Envuelve la llamada a `detectar_formato` de `magic_bytes.py`. Recibe la secuencia de bytes resultante de un intento de descifrado y devuelve el formato detectado, o `None` si no coincide ninguna firma conocida.
- **resultado(cifrado, llave, formato, ruta):** Imprime en consola el resumen del descifrado exitoso: el nombre del algoritmo utilizado, la llave encontrada, el formato del archivo recuperado y la ruta donde fue guardado.

Descifrado por fuerza bruta

Cada método de descifrado se implementó de manera independiente. El espacio de búsqueda de cada método es el siguiente:

- **Base64** (`base64_fuerza_bruta`): No hay llave que iterar. Se interpreta el contenido del archivo como texto ASCII, se decodifica con `base64_decode` y se verifica el resultado.
- **César** (`cesar_fuerza_bruta`): El cifrado César sobre bytes aplica el corrimiento $C_i = (P_i + k) \bmod 256$, por lo que el espacio de llaves es $k \in \{0, 1, \dots, 255\}$, es decir, 256 posibles llaves.
- **Decimado** (`decimado_fuerza_bruta`): El cifrado de decimación opera como $C_i = (a \cdot P_i) \bmod 256$. Para que la función sea invertible, a debe ser coprimo con 256 (es decir, $\gcd(a, 256) = 1$). Dado que $256 = 2^8$, los valores válidos de a son los impares en $\{1, \dots, 255\}$, lo que da $\phi(256) = 128$ llaves posibles. Estos valores se precomputan al inicio del programa en la lista `LLAVES_DECIMADO`.
- **Afín** (`afin_fuerza_bruta`): El cifrado afín combina los dos anteriores: $C_i = (a \cdot P_i + c) \bmod 256$. El espacio de búsqueda es el producto cartesiano de las 128 llaves válidas de a (coprimas con 256) y los 256 valores posibles de c , lo que resulta en $128 \times 256 = 32\,768$ combinaciones.

Proceso principal

Con todas las implementaciones anteriores, se valida que la ruta recibida corresponde a un archivo existente y se lee su contenido binario. Luego itera sobre los métodos disponibles y ejecuta cada uno en secuencia hasta que uno termine exitosamente.

```

def descifrar_archivo(ruta_cifrado: str) -> None:
    """
    Proceso de descifrado para un archivo dado.

    Prueba cada estrategia en orden hasta encontrar una que funcione.

    Params:
    | ruta_cifrado (str): Ruta al archivo cifrado
    """
    ruta = Path(ruta_cifrado)

    if not ruta.is_file():
        print(f"No se encontro el archivo")
        sys.exit(1)

    nombre_base = ruta.stem # nombre sin extension

    print(f"\nArchivo : {ruta}\n")

    datos = leer_archivo(str(ruta))

    exito = False
    for metodo in METODOS_CIFRADO:
        if metodo(datos, nombre_base):
            exito = True
            break

    if exito:
        print("Archivo descifrado con exito.")
    else:
        print("No se pudo descifrar el archivo con ningun metodo de cifrado.")

```

Figura 5: Función principal `descifrar_archivo` que orquesta el proceso.

2.5. Preguntas

1. ¿Cuántos primos relativos hay en $\mathbb{Z}_{256}$?

Primero conviene recordar que dos enteros $a, b \in \mathbb{Z}$ son primos relativos si su máximo común divisor es 1, es decir, si $\gcd(a, b) = 1$.

Por otro lado, $\mathbb{Z}_m$ denota al conjunto de todas las clases de equivalencia diferentes módulo m . Donde cada clase de equivalencia se define como $[z]_m = \{x \in \mathbb{Z} \mid z \equiv x \pmod{m}\}$. De este modo, cuando hablamos de $\mathbb{Z}_{256}$ nos referimos a un conjunto con 256 elementos, es decir:

$$\mathbb{Z}_{256} = \{[0]_{256}, [1]_{256}, [2]_{256}, \dots, [255]_{256}\}$$

Después de estos recordatorios, para poder contar los primos relativos en $\mathbb{Z}_{256}$ podemos usar la función indicatriz de Euler. Si $n \in \mathbb{N}$, entonces $\phi(n)$ se define como el número de enteros positivos menores o iguales a n y coprimos con n , es decir, formalmente se puede definir como:

$$\phi(n) = |\{m \in \mathbb{N} \mid 1 \leq m \leq n \wedge \gcd(m, n) = 1\}|$$

Dentro de las propiedades que nos ofrece la función indicatriz de Euler encontramos que:

$$\phi(p^k) = (p - 1)p^{k-1} \text{ si } p \text{ es un número primo y } k \in \mathbb{N}$$

Dado que queremos contar los primos relativos en $\mathbb{Z}_{256}$, conviene ver que $256 = 2^8$. Por lo que $p = 2$ (pues 2 es primo) y $k = 8 \in \mathbb{N}$ en la propiedad que acabamos de enunciar. Aplicando la fórmula:

$$\phi(2^8) = (2 - 1)2^{8-1} = 1 \cdot 2^7 = 128$$

Esto nos muestra que existen 128 primos relativos en $\mathbb{Z}_{256}$.

2. Sea $f : \mathbb{Z}_n \rightarrow \mathbb{Z}_n$ tal que $f(i) = k \cdot i$ para $i \in A$ con A un alfabeto cualquiera. ¿Qué se debe cumplir para que f sea una función biyectiva?

Por definición, para que f sea biyectiva debe cumplirse que sea inyectiva y sobreyectiva al mismo tiempo.

Inyectividad: Para que f sea inyectiva debe cumplirse que no hay dos elementos del dominio que tengan la misma imagen, es decir, para $a, b \in \mathbb{Z}_n$, si $f(a) = f(b)$, entonces $a = b$.

Supongamos primero que para $a, b \in \mathbb{Z}_n$, $f(a) = f(b)$, es decir, $[k \cdot a]_n = [k \cdot b]_n$. Recordando que dos clases de equivalencia son iguales si y sólo si sus representantes son congruentes módulo n ($[x]_n = [y]_n \iff x \equiv y \pmod{n}$) tenemos que $ka \equiv kb \pmod{n}$, lo cual es equivalente a:

$$k(a - b) \equiv 0 \pmod{n}$$

es decir, $n \mid k(a - b)$ (por definición de congruencia). Aquí es importante mencionar que buscamos cancelar el factor k para que se cumpla que $n \mid (a - b)$, que por definición de congruencia implicaría $a \equiv b \pmod{n}$.

Para este propósito, recurrimos al Lema de Euclides donde si $n \mid k \cdot m$ y $\gcd(k, n) = 1$, entonces $n \mid m$.

De esta manera, si se cumple que $\gcd(k, n) = 1$, podemos concluir que $n \mid (a - b)$, es decir, $a \equiv b \pmod{n}$, lo que en $\mathbb{Z}_n$ significa que $[a]_n = [b]_n$, por lo tanto $a = b$ y f es inyectiva.

Observación: La condición $\gcd(k, n) = 1$ es la hipótesis que necesitamos para que el argumento sea válido. Si en cambio $\gcd(k, n) = d > 1$, entonces existe $m = n/d$ con $0 < m < n$ tal que $k \cdot m \equiv 0 \pmod{n}$. Esto se debe a que $d \mid k$ (pues $d = \gcd(k, n)$), por lo que $k = d \cdot q$ para algún $q \in \mathbb{Z}$, y entonces $k \cdot m = d \cdot q \cdot (n/d) = q \cdot n$, que es claramente múltiplo de n . Esto implica $f(m) = [k \cdot m]_n = [0]_n = f(0)$ con $m \neq 0$, por lo que f no sería inyectiva y, por tanto, no sería biyectiva.

Sobreyectividad: Para que f sea sobreyectiva debe cumplirse que todo elemento del codominio $\mathbb{Z}_n$ es imagen de algún elemento del dominio, es decir, para todo $j \in \mathbb{Z}_n$ existe $i \in \mathbb{Z}_n$ tal que $f(i) = j$.

Ahora bien, si $\gcd(k, n) = 1$, entonces k es invertible en $\mathbb{Z}_n$, es decir, existe $k^{-1} \in \mathbb{Z}_n$ tal que $k \cdot k^{-1} \equiv 1 \pmod{n}$. Dado un elemento cualquiera $j \in \mathbb{Z}_n$, basta tomar $i = k^{-1} \cdot j \in \mathbb{Z}_n$, pues:

$$f(i) = k \cdot (k^{-1} \cdot j) = (k \cdot k^{-1}) \cdot j \equiv 1 \cdot j \equiv j \pmod{n}$$

Así, todo elemento de $\mathbb{Z}_n$ tiene preimagen y f es sobreyectiva.

$\therefore$ La condición necesaria y suficiente para que $f(i) = k \cdot i$ sea una función biyectiva en $\mathbb{Z}_n$ es que $\gcd(k, n) = 1$, es decir, que k y n sean primos relativos.

3. ¿Cuántas posibles combinaciones no triviales existen para cifrar bytes con César, Decimado y Afín?
 - **César:** 255 combinaciones no triviales (256 en total, menos la clave trivial $k = 0$).
 - **Decimado:** 127 combinaciones no triviales (128 números impares coprimos con 256, menos la clave trivial $a = 1$).
 - **Afín:** 32,767 combinaciones no triviales ($128 \times 256 = 32,768$ en total, menos la clave trivial $a = 1, b = 0$).
4. ¿Por qué el sistema de archivos de UNIX, aunque un archivo tenga una extensión diferente (o incluso no tenga), sigue reconociendo al archivo original?
 - UNIX inspecciona el interior del archivo usando los primeros bytes para leer su firma digital, la cual indica el tipo de archivo real. También los metadatos y datos físicos residen todavía. El nombre del archivo y su extensión son solo una etiqueta visual para el usuario que apunta a este.
5. ¿Por qué los archivos descifrados tienen exactamente el mismo tamaño que antes de cifrar, pero no pudimos leerlos? ¿Por qué no tuvimos que agregar/quitar nada?

6. Ya que Base64 no es un cifrado, sino codificación, ¿en qué casos podemos usarlo?

3. Conclusiones

4. Referencias